

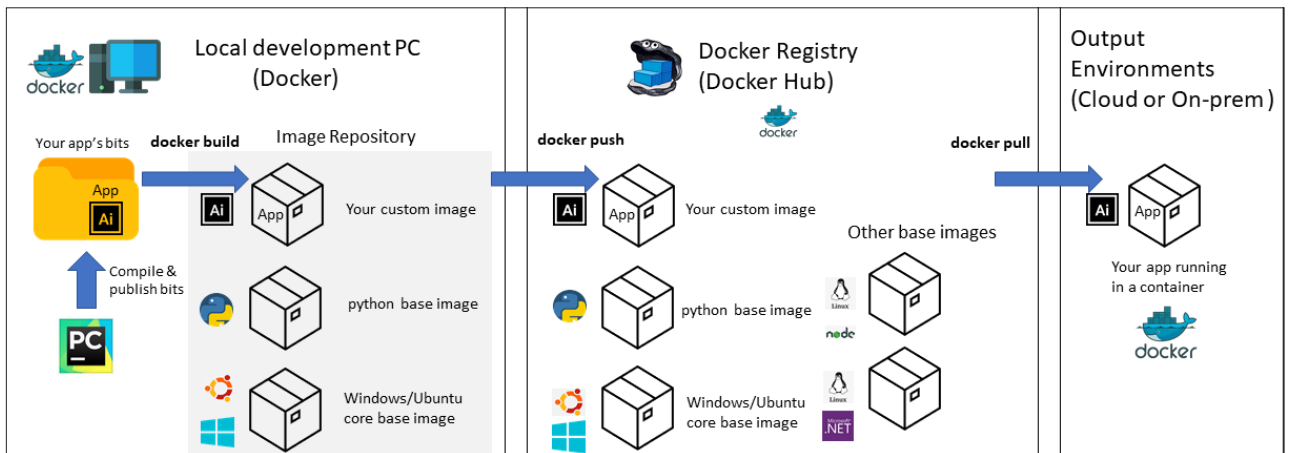


Figure 1: Docker-based workflow

Inside a containerised AI workflow: From code to cloud

In a contemporary containerised artificial intelligence pipeline, development is done locally, where an ML engineer or data scientist develops the application from a root image — Python on Ubuntu or some other Linux OS. The development environment is all the dependencies, libraries (such as TensorFlow, PyTorch), and configuration files bundled in a Docker container. After the environment is in shape, the developer runs a Docker build to establish a personalised image of the application.

The image is pushed into a Docker registry such as Docker Hub or an on-cloud private registry. This renders the image accessible to remote environments and teams, and enables collaborative development, CI/CD pipelines, and model and code version control.

Figure 2 illustrates a typical AI/ML system architecture running inside Docker containers. It showcases a modular frontend (built with React, Vue.js, or Svelte) and a flexible backend (using frameworks like Flask, FastAPI, or Django),

all hosted behind an NGINX reverse proxy. The entire stack runs on a Dockerized infrastructure, orchestrated with Docker Compose. The backend connects to high-performance compute (e.g., NVIDIA A100 GPUs) and integrates with LLMs and vector stores for intelligent processing.

Finally, when deployed, the image is copied from the registry to a production environment — whether a cloud service like AWS/GCP/Azure or an on-prem server. The AI app runs in the same Docker container, under the same environment it was initially coded and tested in. This guarantees consistency of outputs, eliminates dependence complications, and makes scaling and maintenance exponentially easier.

Dockerizing intelligence: Real-world use cases and edge deployments

As AI applications move to distributed, real-time systems from centralised processing environments, the necessity of containerisation becomes more critical. Docker's lightness and platform independence make it a key facilitator of edge

Table 1: Use cases of Dockerized AI in edge deployments

Application	Docker role	Deployment context
Respiratory disease detection via AI from audio data	Containerises inference models for low-resource edge devices	Rural clinics, portable devices
Predictive maintenance for industrial machinery	Hosts data pipelines and ML models for anomaly detection	On-premises edge gateways
Customer footfall and behaviour analytics	Deploys AI vision models with local privacy-preserving inference	Smart shelves, CCTV analytics
Crop monitoring and pest detection	Supports containerised image classifiers on mobile field devices	Drones, IoT sensors in farmlands
Object detection in autonomous vehicles	Enables real-time model inference on GPU-accelerated edge nodes	Drones, self-driving units